

Question 5 Report

Title

Implementation and Performance Analysis of Merge Sort and Quick Sort for a Data Analytics Platform

Aim

To implement Merge Sort and Quick Sort using Divide and Conquer, compare their performance based on execution time, memory usage, and stability, and determine the most suitable algorithm for processing large datasets in a data analytics platform.

Algorithm

Merge Sort:

1. Read input.
2. Divide array recursively.
3. Sort left and right halves.
4. Merge sorted halves.
5. Display sorted array and execution time.

Quick Sort:

1. Read input.
2. Select pivot.
3. Partition array.
4. Recursively sort left and right subarrays.
5. Display sorted array and execution time.

Sample Input

8

45 12 67 23 89 10 56 34

Sample Output

Merge Sort Output:

10 12 23 34 45 56 67 89

Execution Time = 0.000010 seconds

Quick Sort Output:

10 12 23 34 45 56 67 89

Execution Time = 0.000006 seconds

```
Enter number of elements: 7
Enter elements:
1 7 9 5 3 2 4

Merge Sort Output:
1 2 3 4 5 7 9
Execution Time = 0.000011 seconds

Quick Sort Output:
1 2 3 4 5 7 9
Execution Time = 0.000001 seconds

...Program finished with exit code 0
Press ENTER to exit console.
```

Analysis

Parameter	Merge Sort	Quick Sort
Best/Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n \log n)$	$O(n^2)$
Extra Memory	$O(n)$	$O(\log n)$
Stable	Yes	No
In-place	No	Yes

Merge Sort guarantees $O(n \log n)$ time and is stable but requires extra memory. Quick Sort is generally faster, uses less memory, and is suitable for large random datasets, though its worst case is $O(n^2)$.

Result

Both algorithms were implemented successfully. Quick Sort performed better for large datasets due to faster average execution time and lower memory usage, while Merge Sort is preferred when stable sorting is required.